CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 5

Registration Form with Frontend Validation and User-Friendly Error Handling

Student Name	Shreyas
Roll Number	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To design an accessible student registration form for StudentHub with HTML5 input types and robust client-side validation in JavaScript using regular expressions, with clear error messages placed next to each field.

- Collect the nine required fields: name, email, mobile number, password, confirm password, course, year, gender and terms acceptance.
- Use correct HTML5 input types and attributes (required, minlength, maxlength, pattern, autocomplete, inputmode).
- Validate every field in JavaScript with regular expressions and show a specific, helpful message below the field.
- Check password strength with a meter and a checklist of the password rules.
- Make the form usable with a keyboard and a screen reader: labels, fieldsets and legends, ARIA states, focus on the first error.
- Intermediate extension: validate in real time while the user types or changes a value.
- Advanced extension: add a custom CAPTCHA drawn on an HTML5 Canvas.
- Record validation test cases with screenshots of valid and invalid inputs.

2. Problem Definition

Official problem definition: *"Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations."*

The Practical 5 folder started with a registration form that opened as a blank page: a rule hid the page until a login script ran, and that script (and theme.js) were missing. The form validated only when submitted, accepted invalid Indian mobile numbers such as 0000000000, rejected valid passwords containing characters like #, used error colours with too little contrast, and redirected to a login page that did not exist. This practical repairs the page and completes the validation, keeping the original form and validation logic as the starting point.

The practical list asks four key questions; the answers for this implementation are:

Key question	Answer in this practical
Are correct input types and attributes used?	Yes: email, tel, password, select, radio and checkbox controls with required, minlength, maxlength, pattern, inputmode and autocomplete (Section 5).
Are validation errors displayed near the relevant fields?	Yes. Each field has its own message element directly below it, linked with aria-describedby; a summary at the top gives the number of fields to correct (Section 7).
Is password strength checked?	Yes. A meter shows Weak, Medium or Strong and a checklist ticks the five rules as the user types; Strong is exactly the same as valid (Section 8).
Is the form accessible using keyboard and screen-reader-friendly labels?	Every control has a label, the form is grouped with fieldsets and legends, invalid fields get aria-invalid, and focus moves to the first invalid field (Section 10).

3. Tools and Technologies

Tool / technology	Use in this practical
HTML5	register.html: form controls, input types, validation attributes, fieldsets, ARIA attributes
CSS3	assets/css/style.css: layout, error and valid states, strength meter, mobile layout
JavaScript (ES6+)	assets/js/validation.js: regular expressions, event handling, strength meter, Canvas CAPTCHA
HTML5 Canvas API	Drawing the CAPTCHA code with rotation and noise
Microsoft Edge 156.0.4314.50	Running and testing the form; headless mode used for the scripted tests and the screenshots
Python 3.12.0	source/check_page.py (static checks) and source/contrast_check.py (WCAG colour contrast)
Visual Studio Code, Git	Source code editor; version control of the StudentHub repository
Operating system	Windows 11

4. Registration Form Implementation

The form is in Practical-5/register.html. Its styles are in assets/css/style.css and its behaviour in assets/js/validation.js; the page contains no inline CSS or JavaScript, uses system fonts and works offline. The fields are grouped in four fieldsets – Personal details, Academic details, Account and Verification – followed by the terms checkbox and two buttons, Register Account and Clear form.

#	Field	Control	Purpose
1	Full name	input type="text"	Student's name, 3–50 letters
2	Email address	input type="email"	Contact email, e.g. name@charusat.edu.in
3	Mobile number	input type="tel"	10-digit Indian mobile number
4	Gender	3 × input type="radio" in a fieldset	Male, Female or Other
5	Course	select	B.Tech CSE, CE, IT or BCA
6	Year of study	select	1st to 4th year
7	Password	input type="password"	New account password, with strength meter
8	Confirm password	input type="password"	Must match the password
9	Terms acceptance	input type="checkbox"	Agreement to the Terms & Conditions
–	CAPTCHA (extension)	canvas + input type="text"	Type the 5-character code drawn on the canvas

- **Submit:** the Register Account button runs all ten checks. If any fail, a summary appears and focus moves to the first invalid field; if all pass, a success message is shown.
- **No data is sent or saved:** the page is static, so submission is always prevented. The success message says so explicitly; the old redirect to the missing login.html was removed.

- **Clear form:** a reset button clears every value, message and state and draws a new CAPTCHA.

5. HTML5 Input Types and Validation Attributes

Field	HTML5 type and attributes
Full name	type="text", required, minlength="3", maxlength="50", pattern="[A-Za-z][A-Za-z .'\-]{2,49}", autocomplete="name"
Email	type="email", required, maxlength="100", autocomplete="email"
Mobile	type="tel", inputmode="numeric" (number keypad on phones), required, minlength and maxlength="10", pattern="[6-9][0-9]{9}", autocomplete="tel-national"
Gender	type="radio" with the same name, required, inside a fieldset with a legend
Course, Year	select with an empty first option, required
Password	type="password", required, minlength="8", maxlength="64", autocomplete="new-password"
Confirm password	type="password", required, maxlength="64", autocomplete="new-password"
CAPTCHA	type="text", required, minlength and maxlength="5", autocomplete="off", spellcheck="false"
Terms	type="checkbox", required

validation.js sets form.noValidate = true, so the browser's own pop-up messages are replaced by the page's messages. If JavaScript is unavailable, the HTML5 attributes still let the browser check the form. The tests confirmed that Edge applies the pattern attributes (for example, R2D2 and 5876543210 are reported as pattern mismatches).

5.1 register.html – personal details

```
25  <form id="registerForm" method="post">
26      <fieldset>
27          <legend>Personal details</legend>
28          <div class="form-row">
29              <div class="form-group" id="name-group">
30                  <label for="name">Full name <span aria-hidden="true">*</span></label>
31                  <input type="text" id="name" name="name" placeholder="Your full name"
    autocomplete="name"
32                         required minlength="3" maxlength="50" pattern="[A-Za-z][A-Za-z .'\-]{2,49}"
33                         aria-describedby="name-hint name-error">
34                  <p class="hint" id="name-hint">3–50 letters; spaces, dots, apostrophes and hyphens
    allowed.</p>
35                  <p class="error-msg" id="name-error"></p>
36              </div>
37              <div class="form-group" id="email-group">
38                  <label for="email">Email address <span aria-hidden="true">*</span></label>
39                  <input type="email" id="email" name="email" placeholder="name@charusat.edu.in"
    autocomplete="email"
40                         required maxlength="100" aria-describedby="email-error">
41                  <p class="error-msg" id="email-error"></p>
42              </div>
43          </div>
44
45          <div class="form-row">
46              <div class="form-group" id="mobile-group">
47                  <label for="mobile">Mobile number <span aria-hidden="true">*</span></label>
48                  <input type="tel" id="mobile" name="mobile" placeholder="9876543210" autocomplete="tel-
    national"
49                         inputmode="numeric" required minlength="10" maxlength="10" pattern="[6-9][0-9]
    {9}"
50                         aria-describedby="mobile-hint mobile-error">
```

5.2 register.html – gender fieldset

```
<div class="form-group" id="gender-group">
    <fieldset class="radio-group" aria-describedby="gender-error">
        <legend>Gender <span aria-hidden="true">*</span></legend>
        <div class="gender-options">
            <label><input type="radio" name="gender" value="Male" required> Male</label>
            <label><input type="radio" name="gender" value="Female"> Female</label>
            <label><input type="radio" name="gender" value="Other"> Other</label>
        </div>
    </fieldset>
    <p class="error-msg" id="gender-error"></p>
</div>
```

6. JavaScript Validation Using Regular Expressions

Field	Regular expression	Meaning
Name	/^[A-Za-z][A-Za-z .'-]{2,49}$/	Starts with a letter; 3–50 letters, spaces, dots, apostrophes or hyphens (checked after trimming spaces)
Email	/^(?!.*\.\.)[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$/	A local part, @, a domain with at least one dot and a top-level domain of 2+ letters; no ".." anywhere
Mobile	/^[6-9]\d{9}$/	Exactly 10 digits, the first one 6, 7, 8 or 9 (Indian mobile numbers)
Password	/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9\s])\S{8,64}$/	Lookaheads require a lowercase letter, an uppercase letter, a digit and a special character; 8–64 characters with no spaces

Each field has its own validator function that returns an error message, or an empty string when the value is valid. The mobile validator checks step by step, so the message explains the actual problem: digits only, then exactly 10 digits, then the first digit. The CAPTCHA comparison converts the input to upper case, so it is not case-sensitive.

6.1 validation.js – regular expressions

```
// Regular expressions
const PATTERNS = {
    name: /^[A-Za-z][A-Za-z .'-]{2,49}$/,                                   // 3–50 chars, starts with a letter
    email: /^(?!.*\.\.)[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$/,
    mobile: /^[6-9]\d{9}$/,                                                 // Indian mobile: 10 digits, starts 6–9
    password: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9\s])\S{8,64}$/,
};
```

6.2 validation.js – one validator per field

```
// One validator per field: returns an error message, or '' when the value is valid
const validators = {
    name: () => {
        const v = value('name').trim();
        if (!v) return 'Please enter your full name.';
        return PATTERNS.name.test(v) ? '' : 'Use 3–50 letters; only spaces, dots, apostrophes and hyphens are allowed.';
    },
    email: () => {
        const v = value('email').trim();
        if (!v) return 'Please enter your email address.';
        return PATTERNS.email.test(v) ? '' : 'Enter a valid email address, for example name@charusat.edu.in.';
    },
    mobile: () => {
        const v = value('mobile').trim();
        if (!v) return 'Please enter your mobile number.';
        if (!/^\d+$/.test(v)) return 'Use digits only.';
        if (v.length !== 10) return 'The mobile number must have exactly 10 digits.';
        return PATTERNS.mobile.test(v) ? '' : 'An Indian mobile number starts with 6, 7, 8 or 9.';
    },
    gender: () => (radios.some((r) => r.checked) ? '' : 'Please select your gender.'),
    course: () => (value('course') ? '' : 'Please select your course.'),
    year: () => (value('year') ? '' : 'Please select your year of study.'),
    password: () => {
        const v = value('password');
        if (!v) return 'Please create a password.';
        return PATTERNS.password.test(v) ? '' : 'The password does not meet all the rules listed above.';
    },
    cPassword: () => {
        const v = value('cPassword');
        if (!v) return 'Please re-enter your password.';
        return v === value('password') ? '' : 'The passwords do not match.';
    },
    captcha: () => {
        const v = value('captcha').trim();
        if (!v) return 'Please type the code shown in the image.';
        return v.toUpperCase() === captchaCode ? '' : 'The code does not match. Try again or press New code.';
    },
    terms: () => (document.getElementById('terms').checked ? '' : 'You must accept the Terms & Conditions.'),
};
```

7. Real-Time Validation and User-Friendly Error Messages

- **On blur:** when the user leaves a text field that has a value, it is checked and marked as “touched”. A field the user only tabbed through is not flagged, so an empty form is not covered in red.
- **On input:** once a field is touched, it is re-checked on every key press or paste, so an error disappears as soon as it is corrected (intermediate extension; the input event also covers pasting, which keyup does not).
- **On change:** selects, radio buttons and the terms checkbox are checked as soon as they change.
- **Linked fields:** changing the password re-checks the confirm-password field.
- **On submit:** all fields are checked; a summary such as “Please correct 3 fields marked below.” appears at the top and focus moves to the first invalid field.
- **Messages:** each message is placed directly below its field, says what is wrong and how to fix it (e.g. “An Indian mobile number starts with 6, 7, 8 or 9.”), and is shown in dark red text with a red field border. Valid fields get a green border.

7.1 validation.js – showing the result next to the field

```
// Shows the message next to the field and sets aria-invalid; returns true when valid
const validate = (field) => {
    const message = validators[field]();
    document.getElementById(`${field}-error`).textContent = message;
    const group = document.getElementById(`${field}-group`);
    group.classList.toggle('has-error', Boolean(message));
    group.classList.toggle('is-valid', !message);
    controlsOf(field).forEach((el) => el.setAttribute('aria-invalid', String(Boolean(message))));
    return !message;
};
```

7.2 validation.js – input, change and blur listeners

```
// Real-time validation: input (each key press or paste), change and blur
FIELDS.forEach((field) => {
    controlsOf(field).forEach((el) => {
        const isText = el.tagName === 'INPUT' && ['text', 'email', 'tel',
'password'].includes(el.type);
        if (isText) {
            el.addEventListener('input', () => {
                if (touched.has(field)) validate(field);
            });
            el.addEventListener('blur', () => {
                if (el.value.trim() || touched.has(field)) {                // do not flag a field
the user only tabbed through
                    touched.add(field);
                    validate(field);
                }
            });
        } else {
            el.addEventListener('change', () => {
                touched.add(field);
                validate(field);
            });
        }
    });
});

document.getElementById('password').addEventListener('input', () => {
    updateStrength();
    if (touched.has('cPassword')) validate('cPassword');                    // keep the match check
up to date
});
```

7.3 validation.js – submit

```
// Submit: check every field, then show either the errors or the success message
form.addEventListener('submit', (e) => {
    e.preventDefault();                                                     // static page: never
submit
    FIELDS.forEach((field) => touched.add(field));
    const invalid = FIELDS.filter((field) => !validate(field));

    if (invalid.length) {
        success.hidden = true;
        summary.hidden = false;
        summary.textContent = `Please correct ${invalid.length} ${invalid.length === 1 ? 'field' :
'fields'} marked below.`;
        controlsOf(invalid[0])[0].focus();                                  // focus the first
invalid field
        return;
    }

    summary.hidden = true;
    document.getElementById('success-name').textContent = value('name').trim();
    success.hidden = false;
    success.focus();
});
```

8. Password Strength Meter

The meter and the checklist use the same five rules as the password regular expression, so the meter can never show Strong for a password that the validator rejects:

Rules met (of 5)	Strength shown
0–2	Weak – red bar, one third wide
3–4	Medium – amber bar, two thirds wide (label text in dark amber for contrast)
5 (all rules)	Strong – green bar, full width; the password is valid

The rules are: 8–64 characters with no spaces, a lowercase letter, an uppercase letter, a number, and a special character (any symbol, not only @$!%*?& as in the original code). Each checklist item turns green with a tick when met, and contains hidden text (“met” / “not met”) for screen readers. The strength label is in an aria-live region and is updated only when the level changes, so a screen reader announces Weak, Medium or Strong without repeating on every key press.

8.1 validation.js – rules and meter

```
// The same rules drive the checklist, the strength meter and PATTERNS.password
const PASSWORD_RULES = [
    { id: 'rule-length', test: (v) => /^\S{8,64}$/.test(v) },
    { id: 'rule-lower', test: (v) => /[a-z]/.test(v) },
    { id: 'rule-upper', test: (v) => /[A-Z]/.test(v) },
    { id: 'rule-digit', test: (v) => /\d/.test(v) },
    { id: 'rule-special', test: (v) => /[^A-Za-z0-9\s]/.test(v) },
];
```

```
// Password strength meter and rules checklist
const strengthBar = document.getElementById('strength-bar');
const strengthText = document.getElementById('strength-text');
const updateStrength = () => {
    const v = value('password');
    const met = PASSWORD_RULES.filter((rule) => {
        const ok = rule.test(v);
        const item = document.getElementById(rule.id);
        item.classList.toggle('met', ok);
        item.querySelector('.rule-state').textContent = ok ? 'met' : 'not met';
        return ok;
    }).length;
    let level = '';
    if (v) level = met === PASSWORD_RULES.length ? 'strong' : met >= 3 ? 'medium' : 'weak';
    if (strengthBar.dataset.level === level) return;                        // announce only when the level changes
    strengthBar.dataset.level = level;
    strengthText.dataset.level = level;
    strengthText.textContent = level ? `Strength: ${level[0].toUpperCase()}${level.slice(1)}` : '';
};
```

8.2 style.css – meter levels

```
.strength-bar[data-level="weak"] { width: 33%; background: var(--error); }
.strength-bar[data-level="medium"] { width: 66%; background: #d97706; }
.strength-bar[data-level="strong"] { width: 100%; background: var(--success); }
```

9. Canvas CAPTCHA Implementation and Limitations

- **Code:** five random characters from a set without look-alikes (no 0/O or 1/I/L).

- **Drawing:** the canvas is filled, six random noise lines are drawn, then each character is drawn with a small random vertical shift, rotation (±25°) and dark colour, followed by 40 noise dots.
- **Checking:** the typed code is trimmed and compared in upper case, so it is not case-sensitive.
- **New code:** the button draws a new code, clears the input and its message, moves focus to the input and announces “A new CAPTCHA code was drawn.” to screen readers. The old code no longer works.

Limitations (stated on the page and in the README): this CAPTCHA is a client-side educational demonstration, not real security. The code is generated and checked in the browser, so any script on the page can read it, and a bot could bypass it easily. It is also not accessible to blind users, because the code exists only as an image; the canvas has a text alternative that explains this, but it cannot provide the code. A real website would generate and verify the CAPTCHA on the server and offer an accessible alternative, such as an audio or non-visual challenge.

9.1 validation.js – drawing the CAPTCHA

```
32  // Canvas CAPTCHA (client-side demonstration only – the code is visible to any script on the page)
33  const CAPTCHA_CHARS = 'ABCDEFGHJKMNPQRSTUVWXYZ23456789';                      // no 0/O, 1/I/L look-
    alikes
34  const canvas = document.getElementById('captcha-canvas');
35  const ctx = canvas.getContext('2d');
36  let captchaCode = '';
37  const randomInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
38
39  const drawCaptcha = () => {
40      captchaCode = Array.from({ length: 5 }, () => CAPTCHA_CHARS[randomInt(0, CAPTCHA_CHARS.length -
    1)]).join('');
41      ctx.fillStyle = '#f1f5f9';
42      ctx.fillRect(0, 0, canvas.width, canvas.height);
43      for (let i = 0; i < 6; i++) {                                             // noise lines
44          ctx.strokeStyle = `hsl(${randomInt(0, 360)}, 45%, 65%)`;
45          ctx.lineWidth = 1.5;
46          ctx.beginPath();
47          ctx.moveTo(randomInt(0, canvas.width), randomInt(0, canvas.height));
48          ctx.lineTo(randomInt(0, canvas.width), randomInt(0, canvas.height));
49          ctx.stroke();
50      }
51      ctx.font = 'bold 30px Consolas, "Courier New", monospace';
52      ctx.textBaseline = 'middle';
53      [...captchaCode].forEach((ch, i) => {                                     // each character
    slightly moved and rotated
54          ctx.save();
55          ctx.translate(28 + i * 36, 30 + randomInt(-5, 5));
56          ctx.rotate((randomInt(-25, 25) * Math.PI) / 180);
57          ctx.fillStyle = `hsl(${randomInt(200, 260)}, 60%, 25%)`;
58          ctx.fillText(ch, -9, 0);
59          ctx.restore();
60      });
61      for (let i = 0; i < 40; i++) {                                            // noise dots
62          ctx.fillStyle = 'rgba(51, 65, 85, 0.35)';
63          ctx.fillRect(randomInt(0, canvas.width), randomInt(0, canvas.height), 2, 2);
64      }
65  };
```

9.2 register.html – canvas and input

```
<label for="captcha">Type the code shown in the image <span aria-hidden="true">*</span></label>
<div class="captcha-row">
    <canvas id="captcha-canvas" width="200" height="60" role="img"
            aria-label="CAPTCHA image with a 5-character code. This visual check cannot be read by screen readers."></canvas>
    <button type="button" class="btn-secondary" id="captcha-refresh">New code</button>
</div>
<input type="text" id="captcha" name="captcha" autocomplete="off" autocapitalize="characters" spellcheck="false"
       required minlength="5" maxlength="5" aria-describedby="captcha-hint captcha-error">
<p class="hint" id="captcha-hint">Not case-sensitive. Demonstration only: the code is checked in the browser, so it does not provide real security.</p>
<p class="error-msg" id="captcha-error"></p>
```

10. Accessibility Features

- **Labels:** every input and select has a visible label connected with for/id; the radio buttons are wrapped in their labels.
- **Grouping:** four fieldsets with legends (Personal details, Academic details, Account, Verification); the gender radio buttons have their own fieldset with the legend “Gender”.
- **Required fields:** marked with a visual asterisk, explained at the top of the form, and with the required attribute so screen readers announce them as required.
- **Hints and errors:** hints (e.g. “10 digits, starting with 6, 7, 8 or 9.”) and the error message are linked to the field with aria-describedby, so they are read when the field gets focus.
- **State:** aria-invalid="true" on invalid fields (all three radio buttons for gender); the error is also shown by text, not by colour alone.
- **Announcements:** the error summary has role="alert", the strength label and a hidden announcer are aria-live="polite", and the success message receives focus so it is read.
- **Keyboard:** only native controls are used, the focus order follows the visual order, there is no positive tabindex, focus moves to the first invalid field, and a 3px blue focus outline is visible.
- **Contrast:** all 19 checked colour pairs meet WCAG 2.1 AA (Section 12.2); the original error red (about 3.8:1) and orange “Medium” text (about 2:1) were replaced.
- **Known limitation:** the visual CAPTCHA cannot be completed by blind users (Section 9).

11. Browser Screenshots

All screenshots are genuine renders of Practical-5/register.html in Microsoft Edge 156 (headless), produced with the same scripted inputs used in the tests. The desktop screenshots are 1280px wide; for readability, the report shows the part containing the form, without the empty background on both sides. The full images are in Practical-5/screenshots/. The CAPTCHA code is random, so it differs between screenshots. Valid fields get a green border through a 0.2-second colour transition; in Figures 8 and 9 the capture happened before this transition finished, so those fields still show the default grey border (Figures 2 and 3 show the green border).

11.1 Invalid: empty form submitted

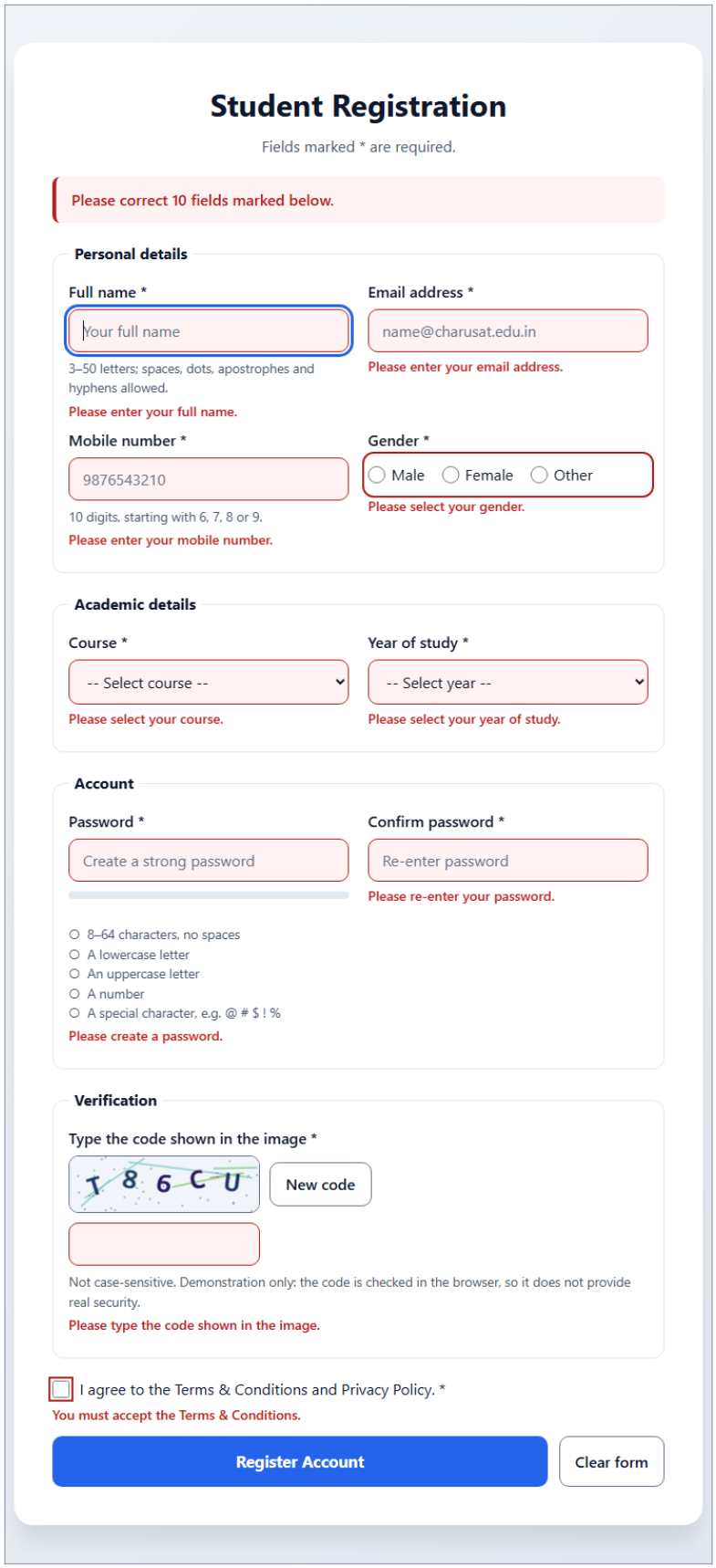


Figure 1: Empty form submitted – summary "Please correct 10 fields marked below.", a message under every field and focus on Full name – empty-errors.png

11.2 Invalid: wrong values in several fields

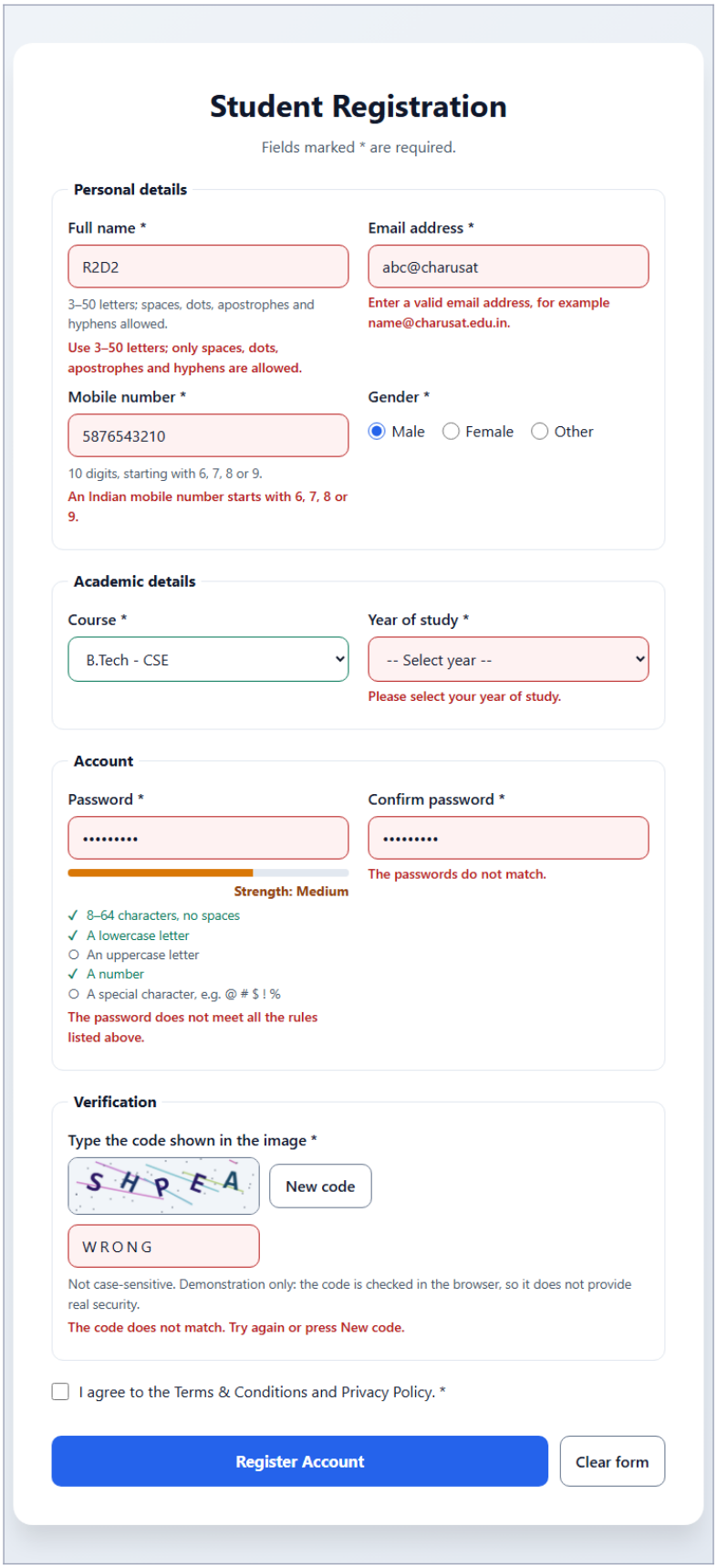


Figure 2: Invalid inputs after leaving each field – name R2D2, email abc@charusat, mobile 5876543210, password1 (Medium, not valid), mismatched confirmation, no year selected and a wrong CAPTCHA; the valid course is marked green – invalid-inputs.png

11.3 Real-time validation

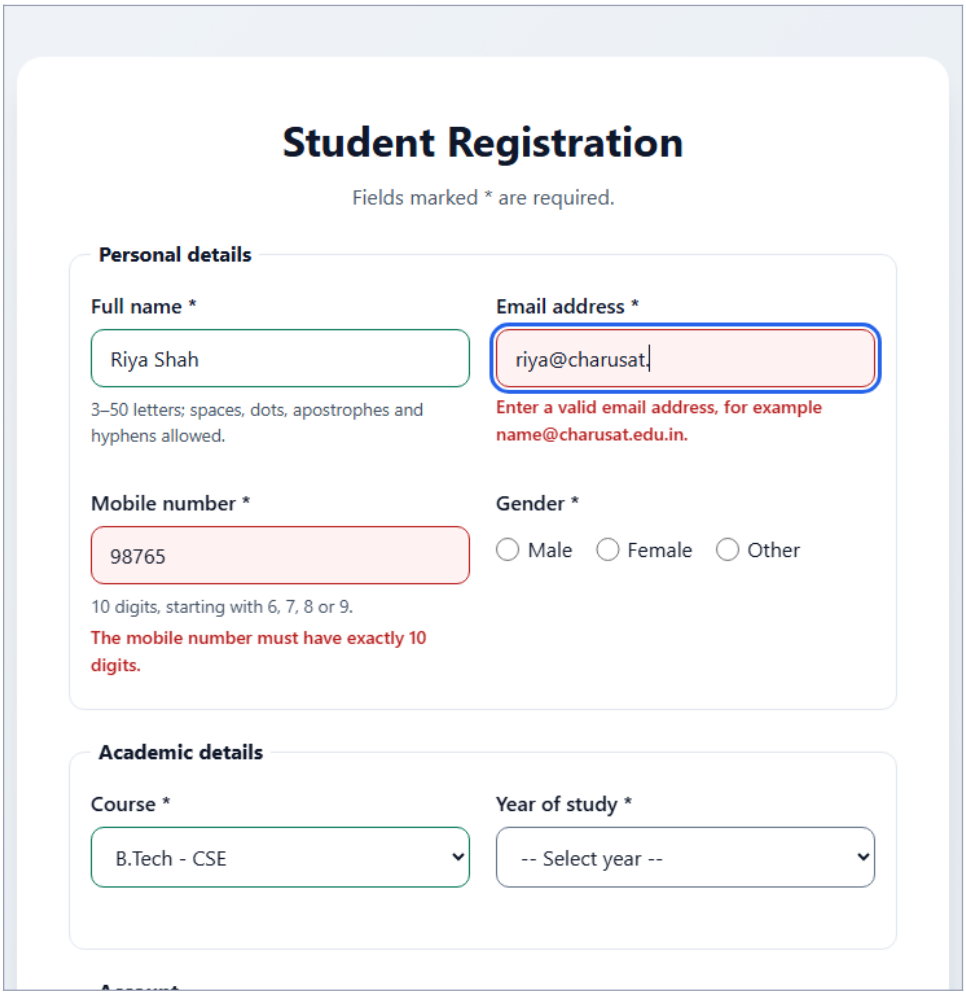


Figure 3: Live validation while typing – Full name and Course are valid (green); the email error stays while the address is incomplete and the mobile number is too short – live-validation.png

11.4 Password strength

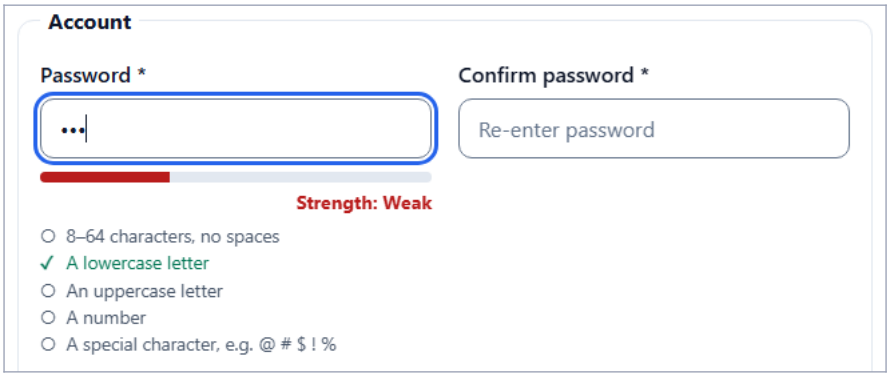


Figure 4: Weak – "abc" meets 1 of 5 rules – password-weak.png

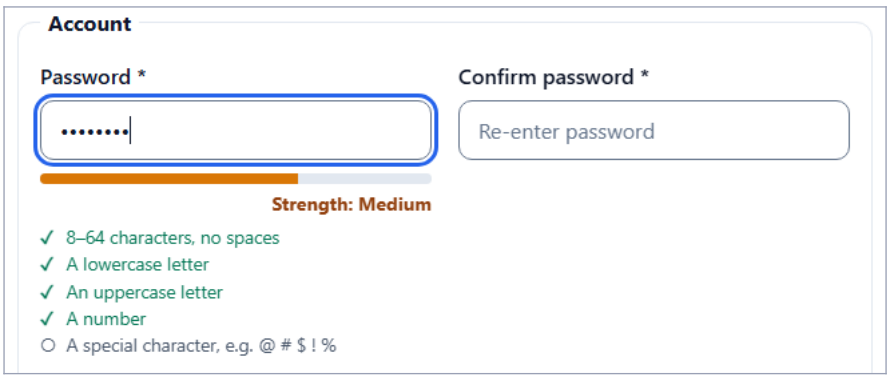


Figure 5: Medium – "Abcdefg1" meets 4 of 5 rules (no special character) – password-medium.png

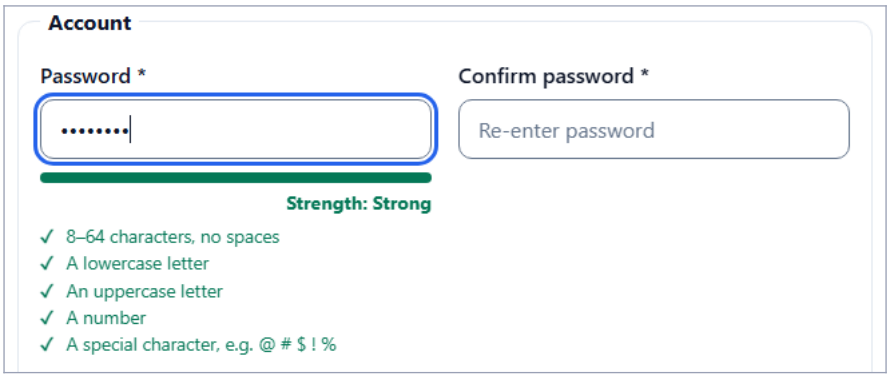


Figure 6: Strong – "Abcdef1#" meets all 5 rules and is valid – password-strong.png

11.5 Canvas CAPTCHA



Figure 7: Invalid – a wrong code is rejected with "The code does not match. Try again or press New code." – captcha-wrong.png

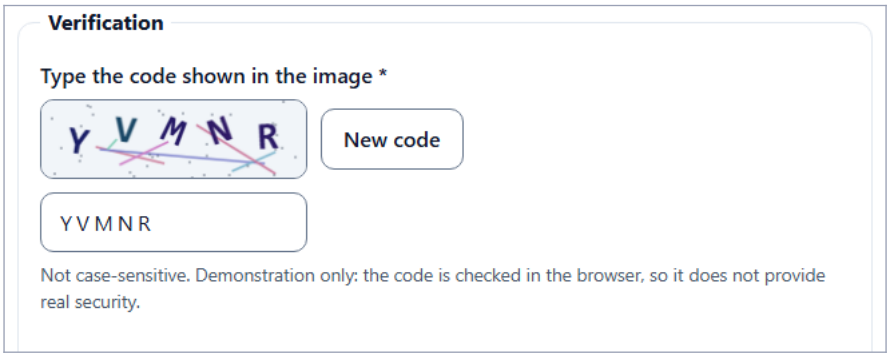


Figure 8: Valid – the code shown on the canvas, typed correctly, is accepted: no error message is shown – captcha-correct.png

11.6 Valid: successful submission

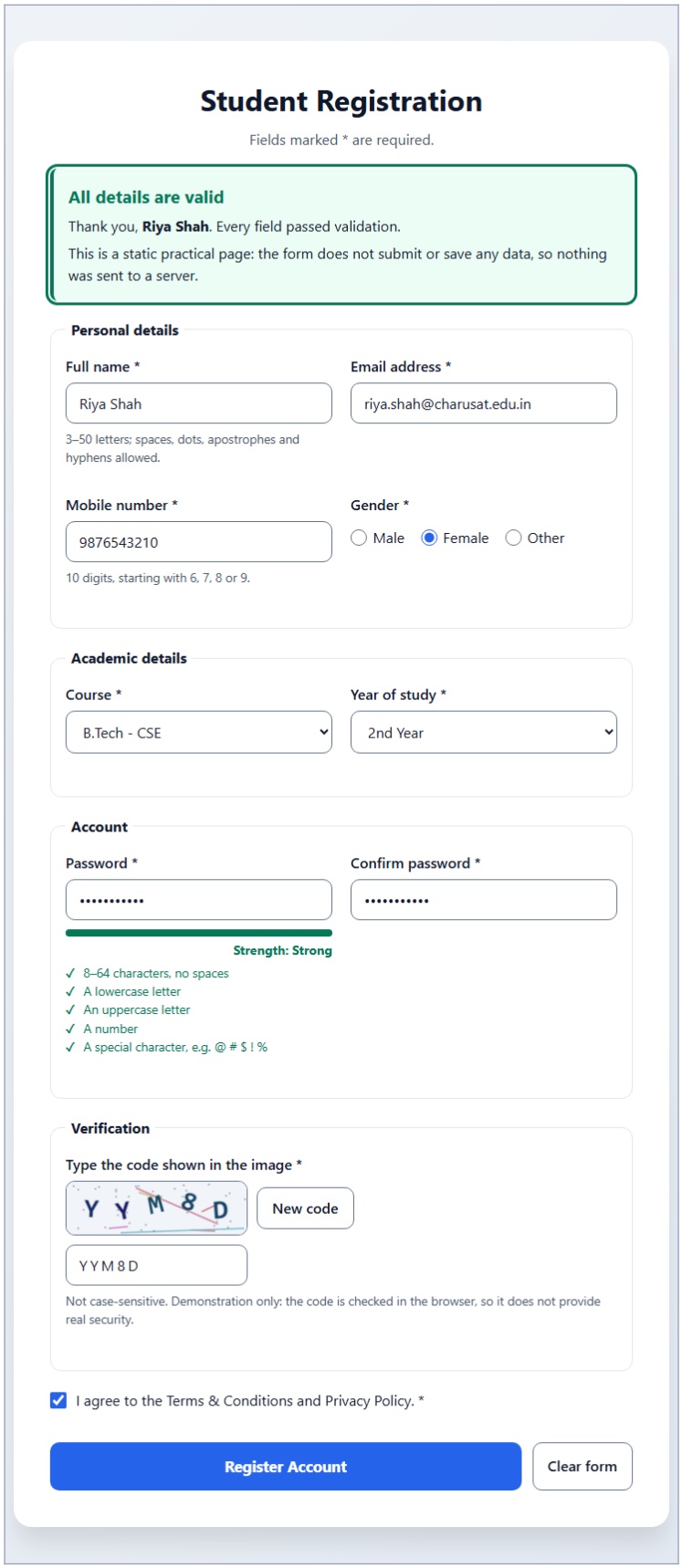
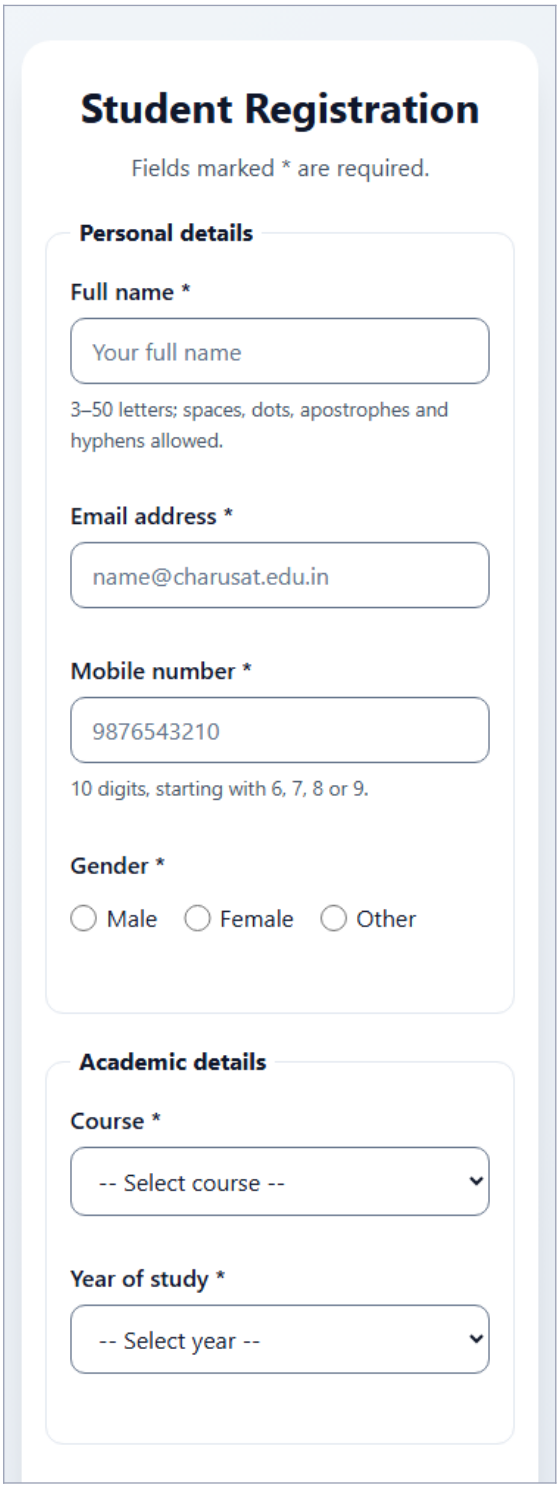
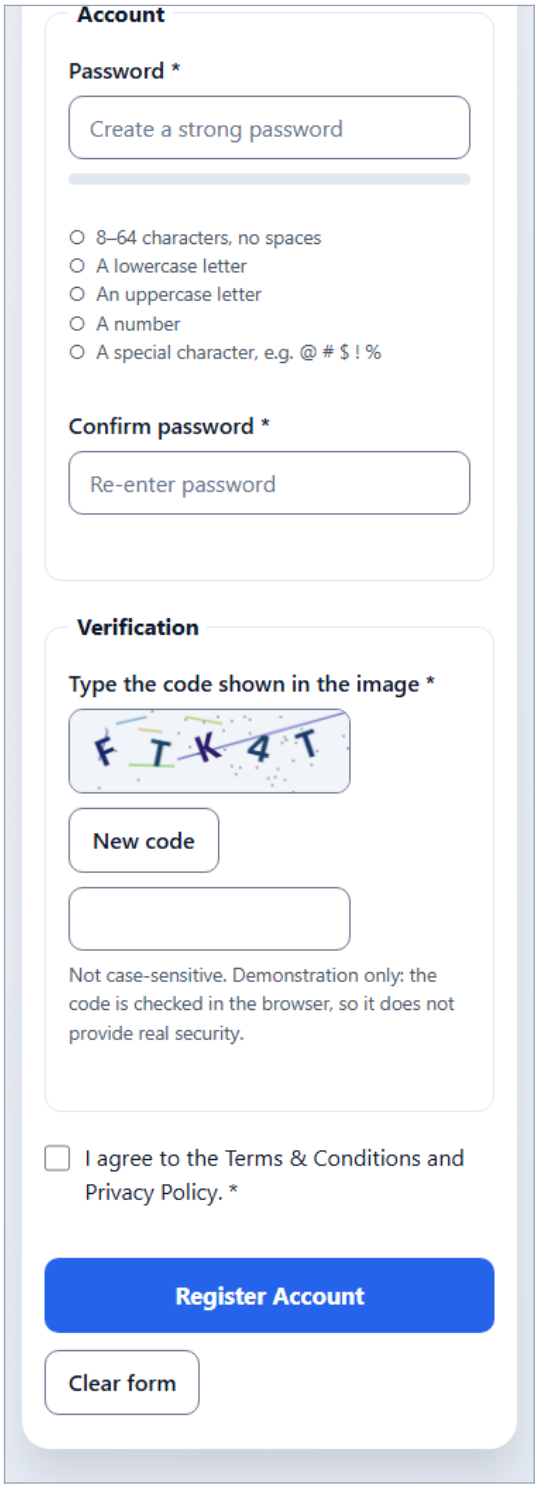


Figure 9: All fields valid – success message focused at the top, stating that nothing was sent or saved; no error messages remain – success.png

11.7 Mobile layout at 375px

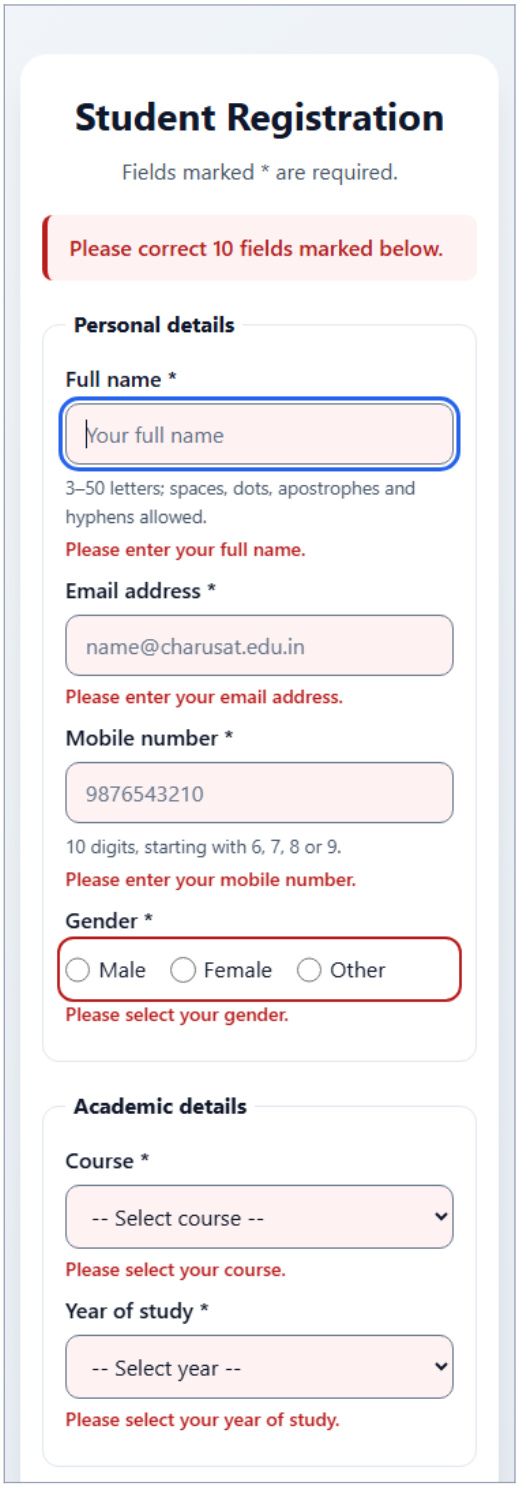


(a) Top half

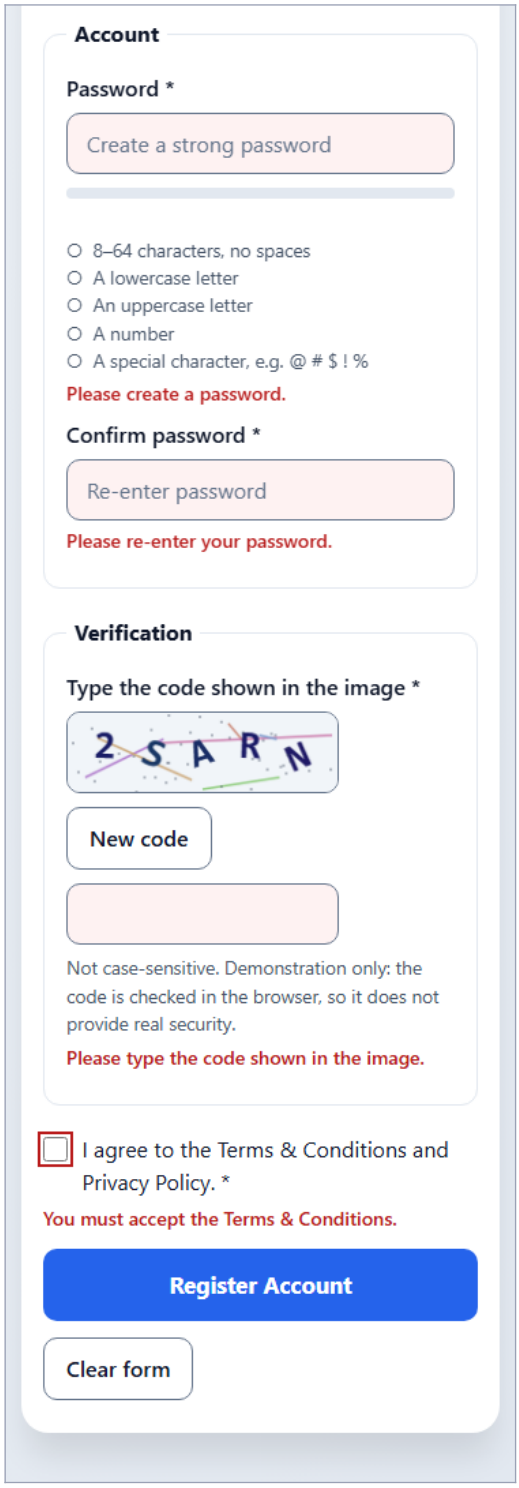


(b) Bottom half

Figure 10: Form at a width of 375px – one column, no horizontal scrolling – mobile-375.png, shown in two halves



(a) Top half



(b) Bottom half

Figure 11: Empty submit at 375px – every message fits below its field – mobile-375-errors.png, shown in two halves

12. Validation Test Cases and Testing Results

12.1 Static checks – source/check_page.py

A Python script written for this practical (standard library only, not the W3C validator) reads register.html, style.css and validation.js. Result: 24 of 24 checks passed.

#	Check	Result
1–3	No inline style attributes, no onclick or other inline handlers, no <style> or inline <script> code	Pass
4–5	No external URLs; all local CSS and JS files exist (no missing auth.js or login.html)	Pass
6–7	No duplicate ids; exactly one h1	Pass
8–11	All nine fields with the right HTML5 types; required on every field; minlength, maxlength, pattern and inputmode; autocomplete values	Pass
12–16	Every control has a label; fieldsets with legends; every aria-describedby target exists; an error element for each of the 10 checks; errors linked with aria-describedby	Pass
17–20	Every id used by validation.js exists; live regions; CAPTCHA canvas with a text alternative and a refresh button; buttons with type and text	Pass
21–24	Regular expressions for name, email, mobile and password; input, change and blur listeners; no alert() or redirect; ES6+ (no var)	Pass

12.2 Colour contrast – source/contrast_check.py

WCAG 2.1 ratios for 19 colour pairs from style.css (4.5:1 for text, 3:1 for borders and the focus outline). All 19 pass. Selected results:

Colour pair	Ratio	Minimum	Result
Error messages (#b91c1c on white)	6.47:1	4.5:1	Pass
Strength: Medium (#92400e on white)	7.09:1	4.5:1	Pass
Strength: Strong / rules met (#047857 on white)	5.48:1	4.5:1	Pass
Placeholder text (#64748b on white)	4.76:1	4.5:1	Pass
Register button (white on #2563eb)	5.17:1	4.5:1	Pass
CAPTCHA characters, lightest colour (#1a4d66 on #f1f5f9)	8.34:1	4.5:1	Pass
Input border (#64748b on white)	4.76:1	3:1	Pass
Focus outline (#2563eb on white)	5.17:1	3:1	Pass

12.3 Edge tests – method

A temporary Python test script (kept outside the submitted files) opened copies of register.html in Microsoft Edge 156 in headless mode. A script added to each copy set values and dispatched real input, change, blur and click events to the page's own listeners, then checked the messages, aria-invalid, focus, the summary and the success message. The CAPTCHA was "read" by recording the characters drawn on the canvas. An error recorder captured page errors, failed file loads and console.error messages in every run. A control run with a deliberately false check confirmed that failures and script errors are reported. Every case also checked for page and console errors. Result: 103 of 103 checks passed, with no console errors.

12.4 Validation test cases – valid and invalid inputs

Field	Invalid inputs tested (error shown)	Valid inputs tested (no error)
Full name	Ab (too short); R2D2 (digits); empty	Riya Shah; Anne-Marie O'Neil; “ Riya Shah ” (spaces trimmed)
Email	abc; abc@; abc@charusat (no dot in domain); a..b@charusat.edu.in	name@charusat.edu.in; first.last+wdf@gmail.com
Mobile	98765 → “must have exactly 10 digits”; 98765abcde → “Use digits only”; 5876543210 → “starts with 6, 7, 8 or 9”; 0000000000; 98765432101	9876543210; 6123456789
Password	abc; password1; Password1 (no special); Pass word1! (space)	Abcdef1#; Str0ng!Pass
Confirm password	Different password → “The passwords do not match.”; empty	Same as the password
Course / Year	“-- Select --” option	B.Tech - IT / 4th Year
Gender	None selected (on submit)	Other
Terms	Unticked	Ticked
CAPTCHA	WRONG; empty; the old code after New code	Correct code in lowercase and in uppercase

12.5 Edge tests – results

Test case	What was checked	Result
Page load	Page visible; stylesheet applied; CAPTCHA drawn; no messages at start; labels; gender fieldset; aria-describedby; HTML5 pattern and required work in Edge	11 / 11
Empty submit	All 10 messages; summary “Please correct 10 fields”; focus on Full name; aria-invalid on every field; success hidden; no page reload	8 / 8
Field values	The 39 valid and invalid inputs in Section 12.4, each checked for the message and aria-invalid	40 / 40
Real-time	No error while first typing; error on blur; cleared while typing; empty field not flagged when tabbed through; selects on change; strength on each key press; confirmation re-checked when the password changes	11 / 11
Password strength	abc and abcdefgh Weak; abcdefg1, Abcdefg1 and “Abc 1#xyzw” Medium; Abcdef1# Strong and valid; checklist and hidden states; empty clears the meter	8 / 8
CAPTCHA	Wrong code rejected; correct code accepted in any case; New code draws a different code, clears the input, focuses it and announces it; old code rejected; a wrong code alone blocks submit	7 / 7
Valid submit	No errors; success message shown and focused; name shown; no navigation and nothing in localStorage, sessionStorage or cookies	5 / 5
Clear form	All messages, states, the summary and the meter cleared; new CAPTCHA	3 / 3
Keyboard order	Focusable controls in visual order; no positive tabindex; native buttons; submit focuses the first invalid field (email; the first gender radio)	6 / 6
Mobile 375px	No horizontal scrolling, with and without all errors; every field fits the screen	4 / 4

12.6 Problems found and fixed during testing

- **Focus outline on the success message:** the browser's default black outline appeared when the message received focus; a green outline matching the message was added.
- **Strength bar in screenshots:** headless Edge sometimes captured the bar mid-animation, so the 0.3-second width animation is switched off in the screenshot copies only; the tests run on the unchanged page.

12.7 Not tested

- **Screen readers:** no screen reader was used; the ARIA attributes were checked in the code and the DOM only.
- **Manual keyboard testing:** nobody operated the form with a physical keyboard; events were dispatched by script, and real Tab-key movement was not simulated.
- **Other browsers and devices:** only Microsoft Edge on Windows 11 was used; the 375px tests used a narrow browser frame, not a real phone.
- **W3C validation:** the page was not submitted to the W3C Markup Validation Service.

13. Results and Conclusion

The StudentHub registration form now works offline and validates all nine required fields – name, email, mobile number, password, confirm password, course, year, gender and terms acceptance – using HTML5 input types and attributes together with JavaScript regular expressions. Each error message is specific and appears directly below its field, the password strength meter and checklist follow the same rules as the validator, and a successful submission shows an on-page message that states clearly that no data is sent or saved.

Both extensions are included: real-time validation on input, change and blur events (intermediate), and a Canvas CAPTCHA with a refresh button and case-insensitive checking (advanced), documented honestly as a client-side demonstration that is neither secure nor accessible to blind users. The static checks (24 of 24), the contrast checks (19 of 19) and the scripted Edge tests (103 of 103, with no console errors) all passed, and the screenshots show both invalid and valid inputs. Client-side checks improve the user experience, but a real registration system must repeat all validation on the server.

14. References

- ITUE203 Web Development Frameworks – Practical List 2026-27, Practical 5: "Registration Form with Frontend Validation and User-Friendly Error Handling", CSPIT, CHARUSAT.
- MDN – Form validation: https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation
- MDN – Regular expressions: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions
- MDN – Canvas API: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- W3C WAI – Forms Tutorial: https://www.w3.org/WAI/tutorials/forms/
- W3C, Web Content Accessibility Guidelines (WCAG) 2.1: https://www.w3.org/TR/WCAG21/
- StudentHub repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023